

```
# Team Name - Team Alpha
# Team Leader - Prabhav Kansal
# Team Member - Priyanshu Kansal
# Team Member - Sagar Gupta
# Project Name - Medical Diagnosis using ANN
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df1=pd.read_csv('breast-cancer.csv')
df=df1.copy()

df.head()

{"type": "dataframe", "variable_name": "df"}

df.isna().sum()
```

id	0
diagnosis	0
radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0
perimeter_se	0
area_se	0
smoothness_se	0
compactness_se	0
concavity_se	0
concave points_se	0
symmetry_se	0
fractal_dimension_se	0
radius_worst	0
texture_worst	0
perimeter_worst	0
area_worst	0
smoothness_worst	0
compactness_worst	0
concavity_worst	0
concave points_worst	0
symmetry_worst	0

```

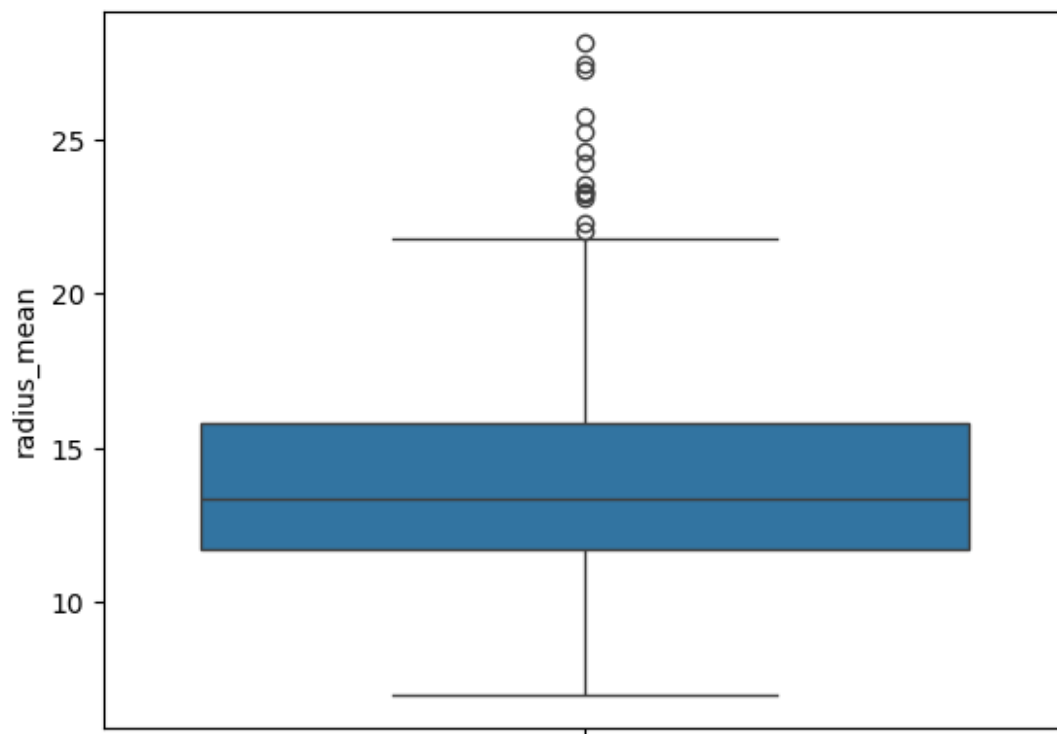
fractal_dimension_worst      0
Unnamed: 32                  569
dtype: int64

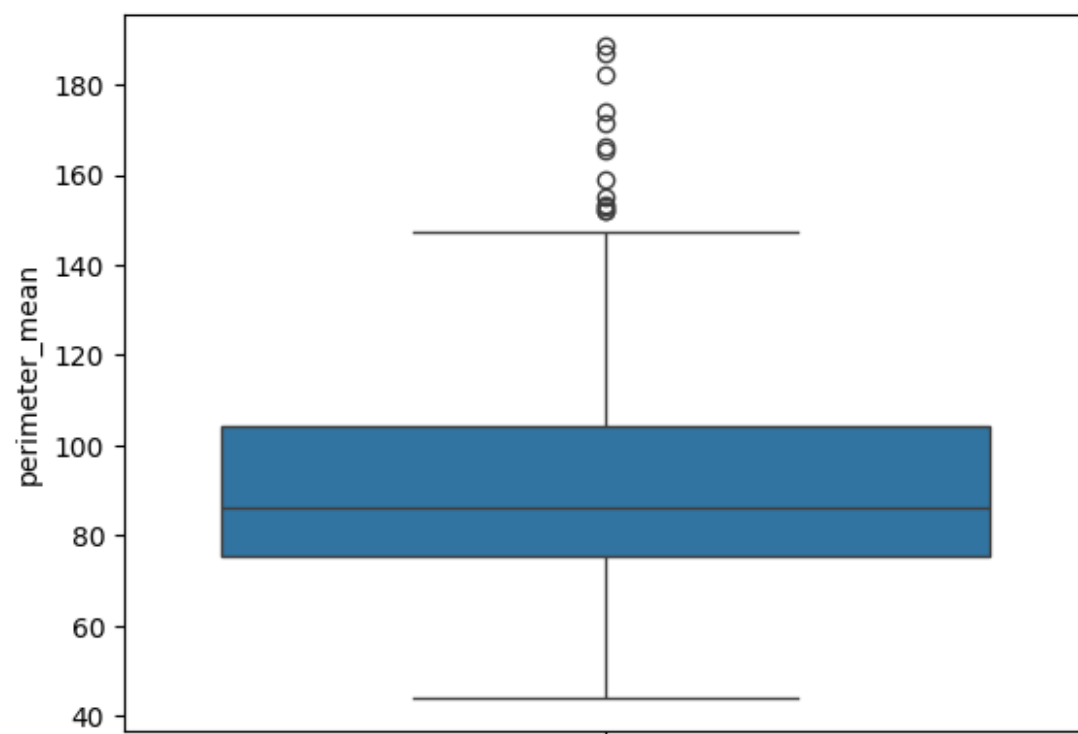
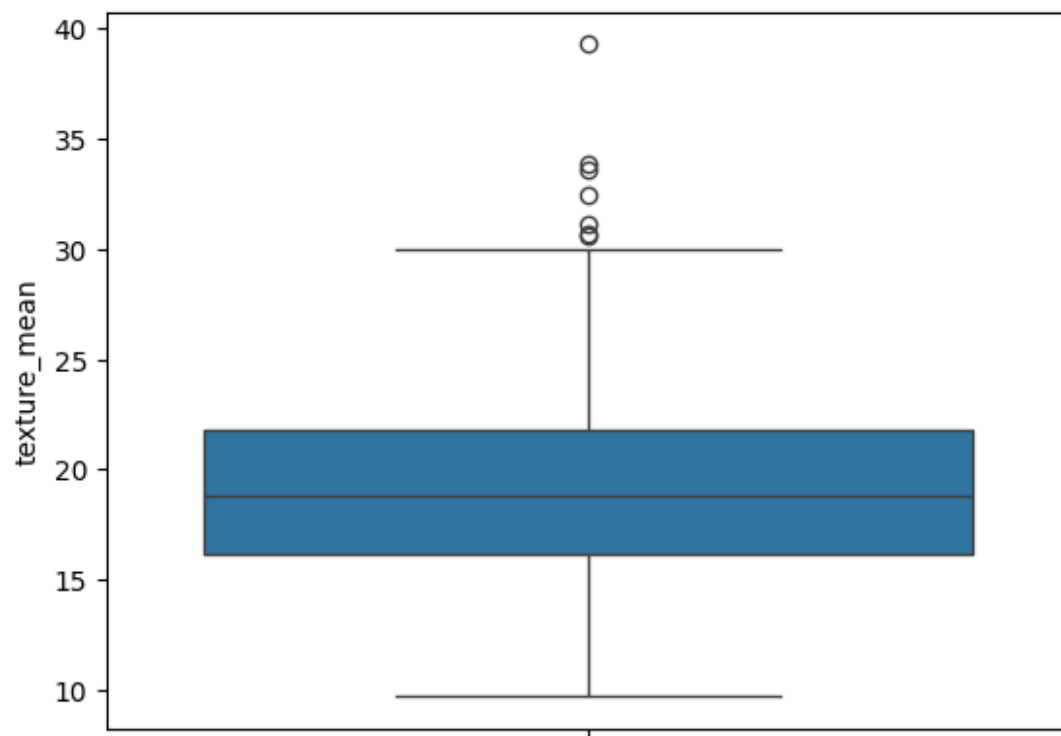
df = df.drop(['id', 'Unnamed: 32'], axis=1)
df.head()

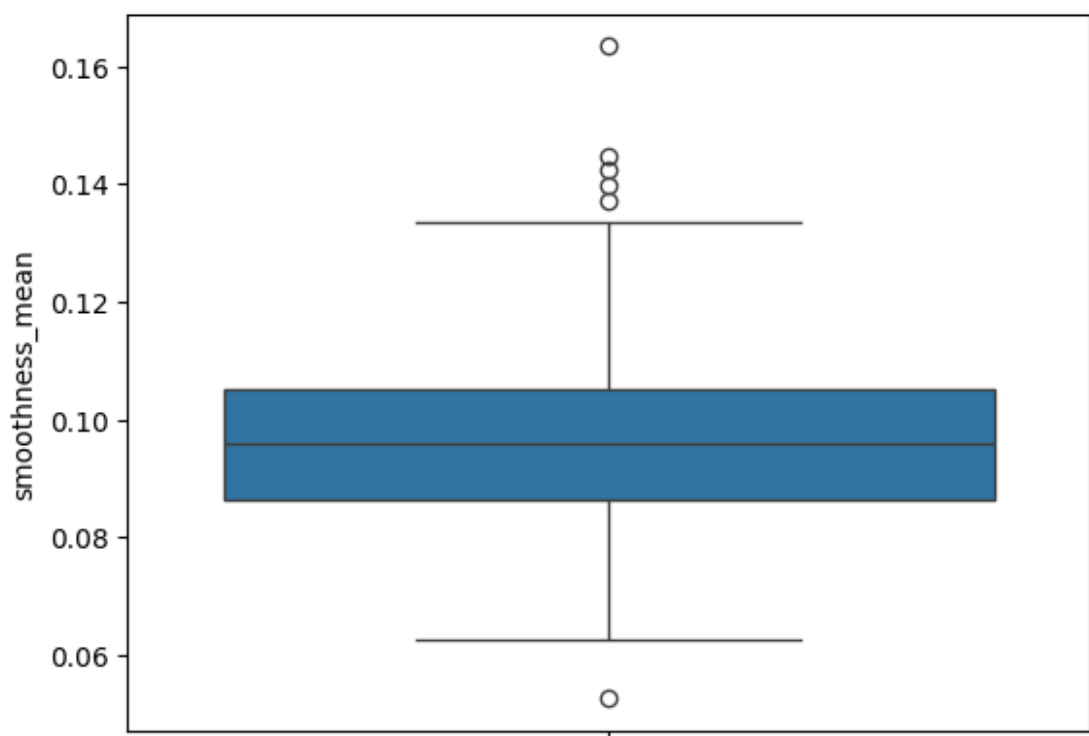
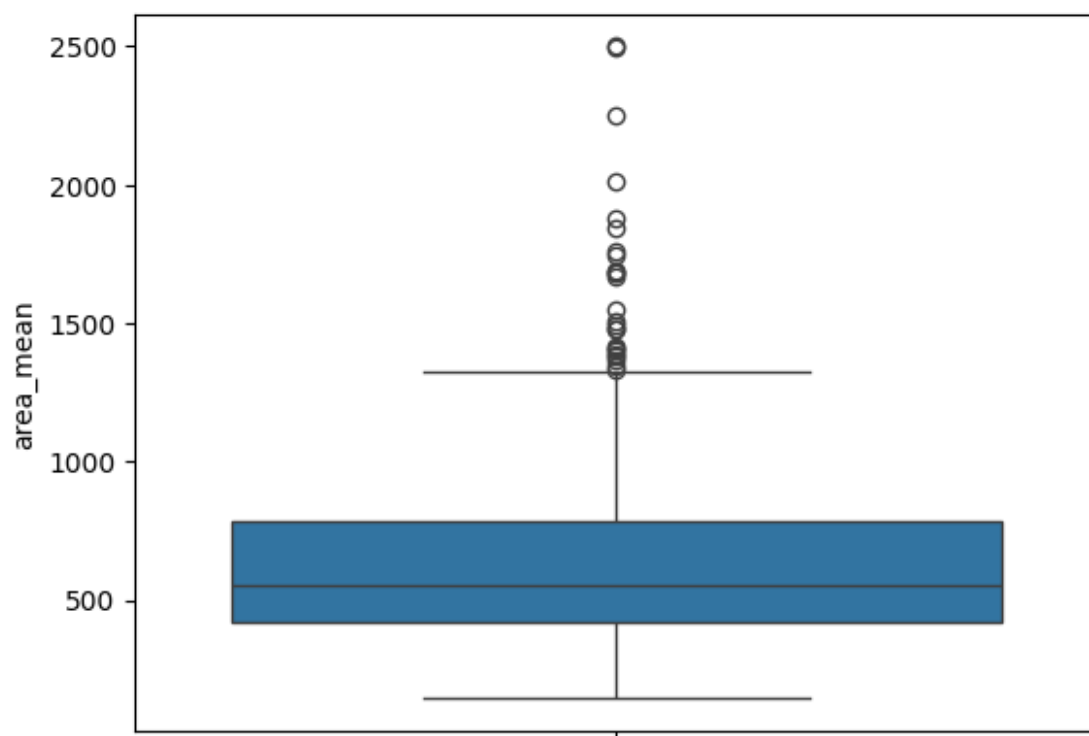
{"type": "dataframe", "variable_name": "df"}

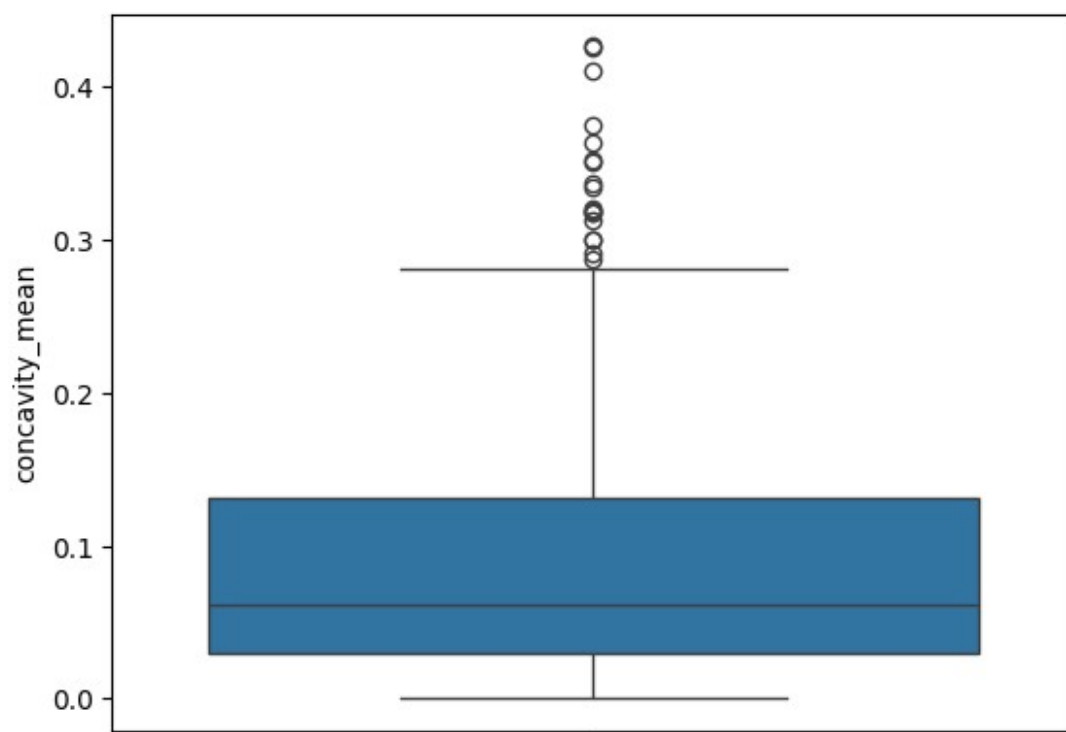
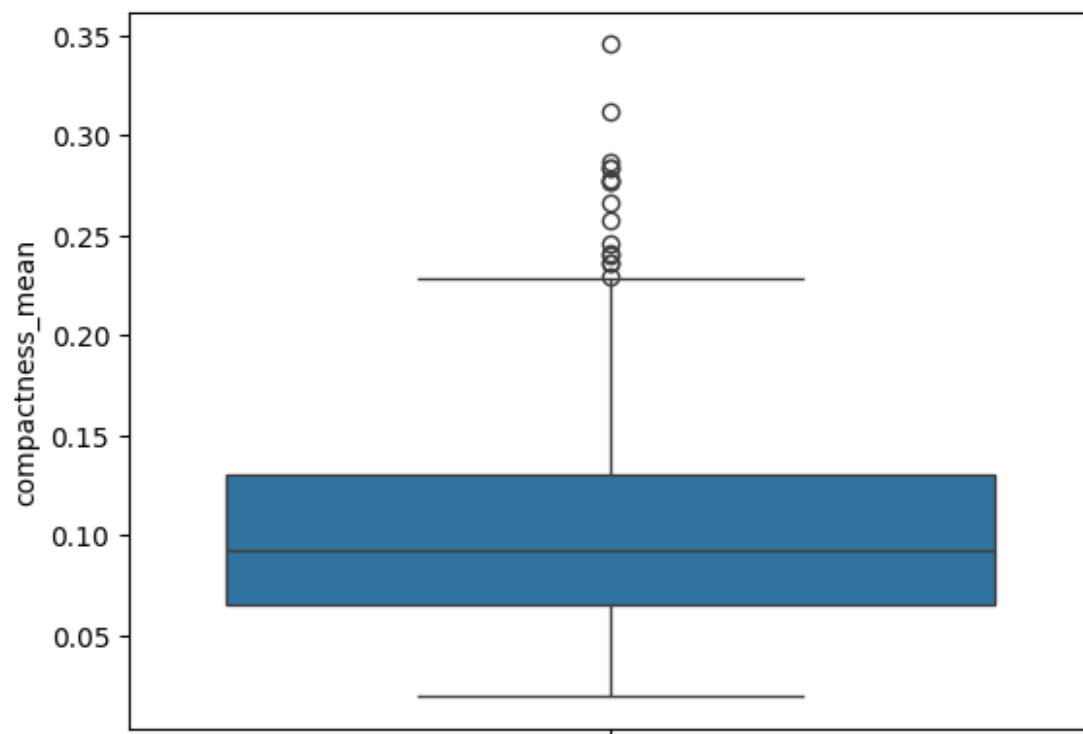
for i in df.select_dtypes(include='number').columns:
    sns.boxplot(df[i])
    plt.show()

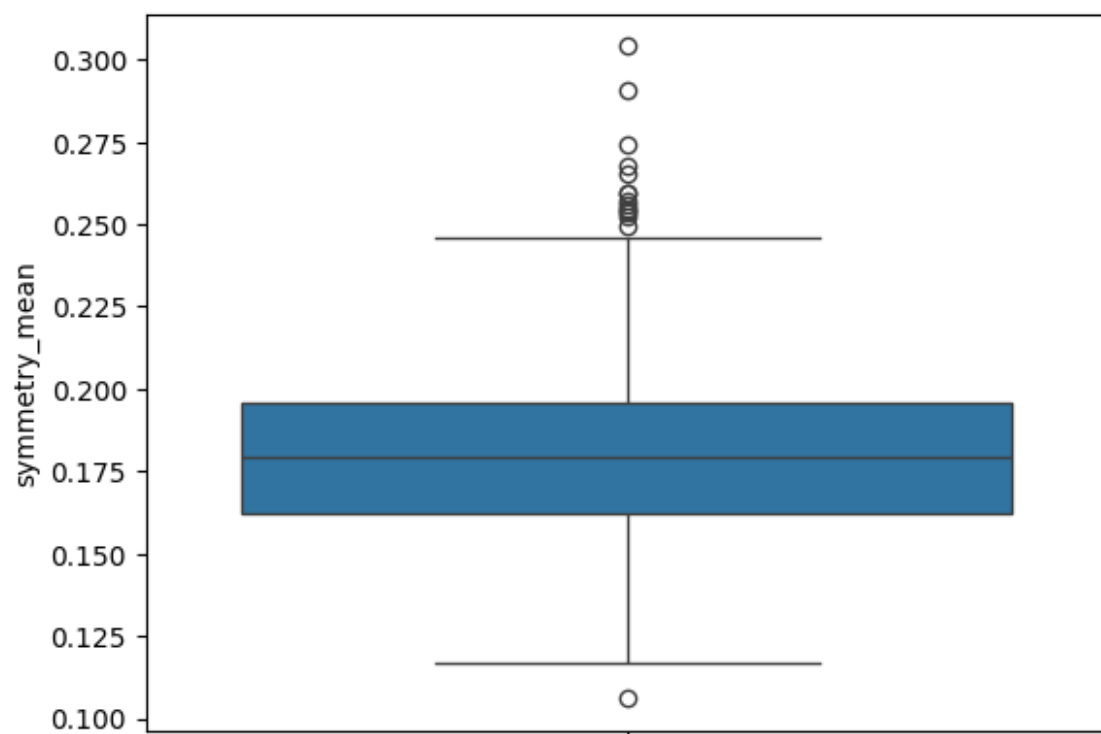
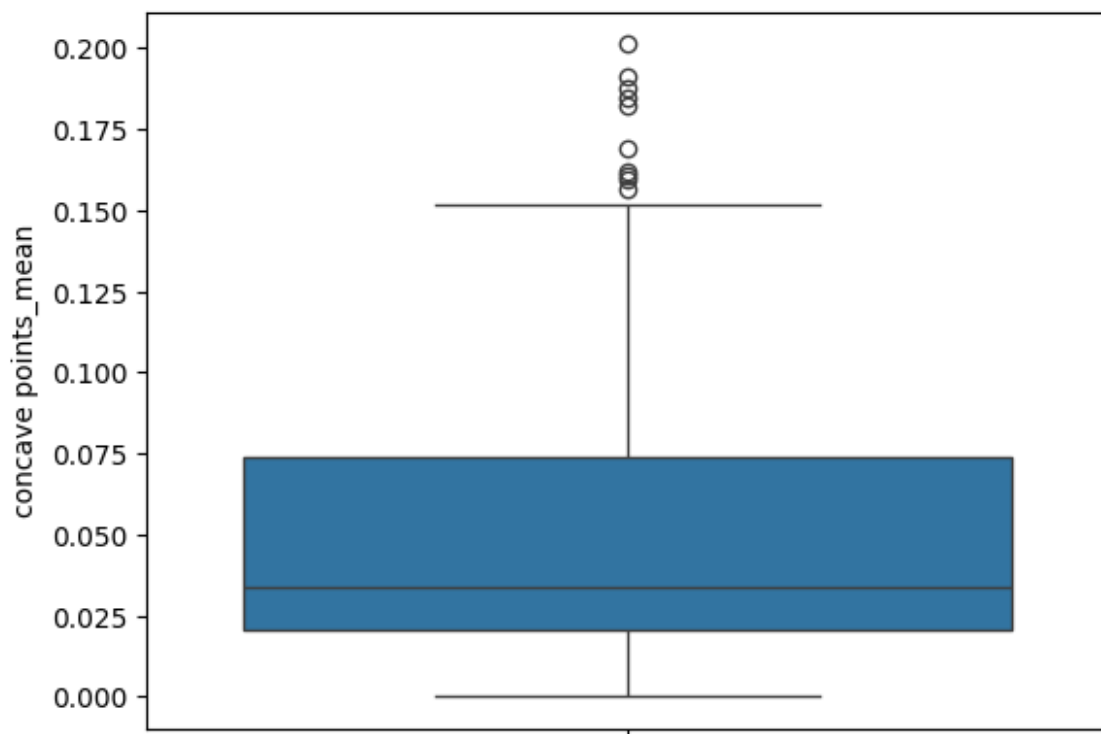
```

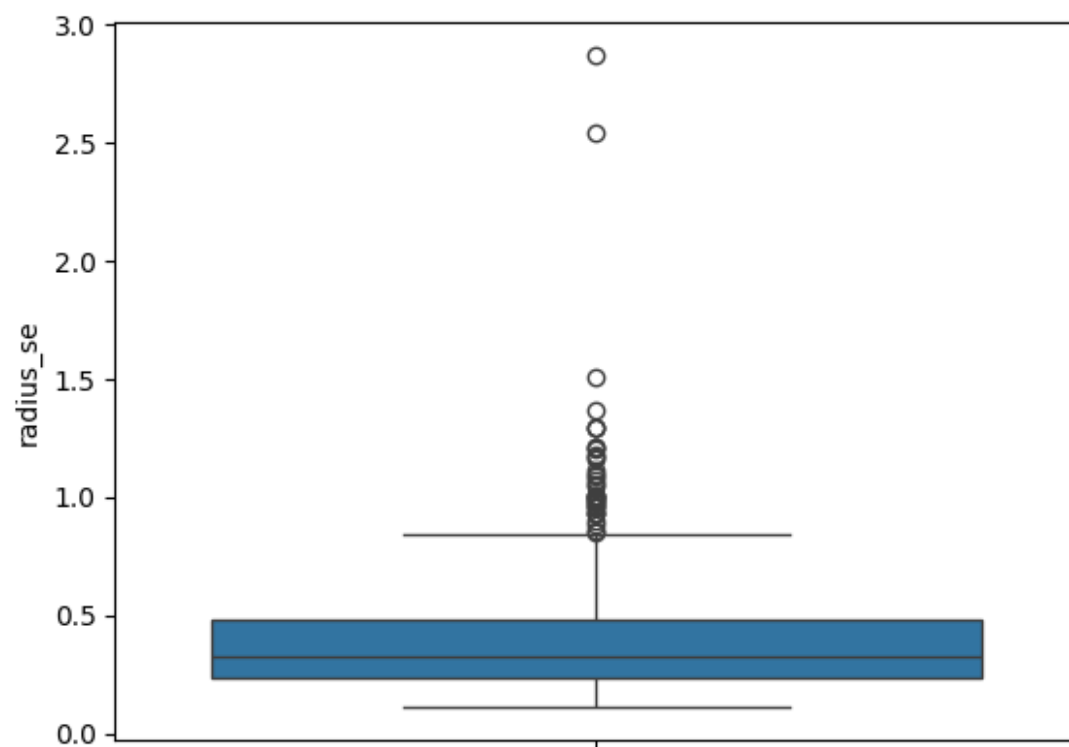
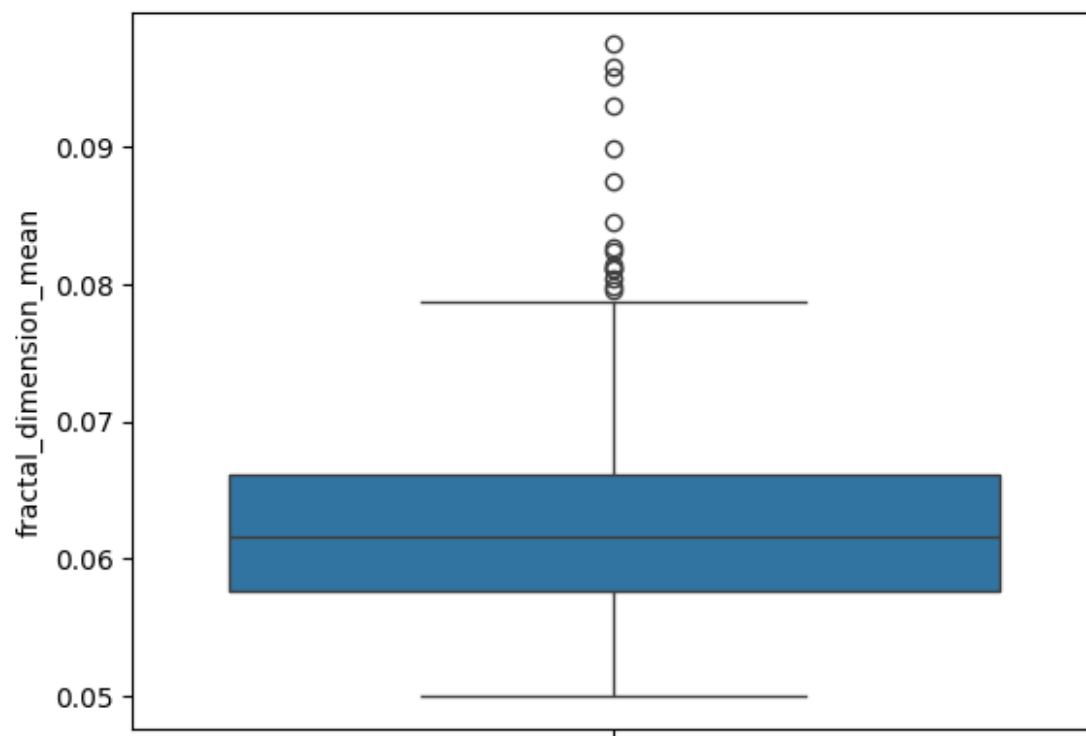


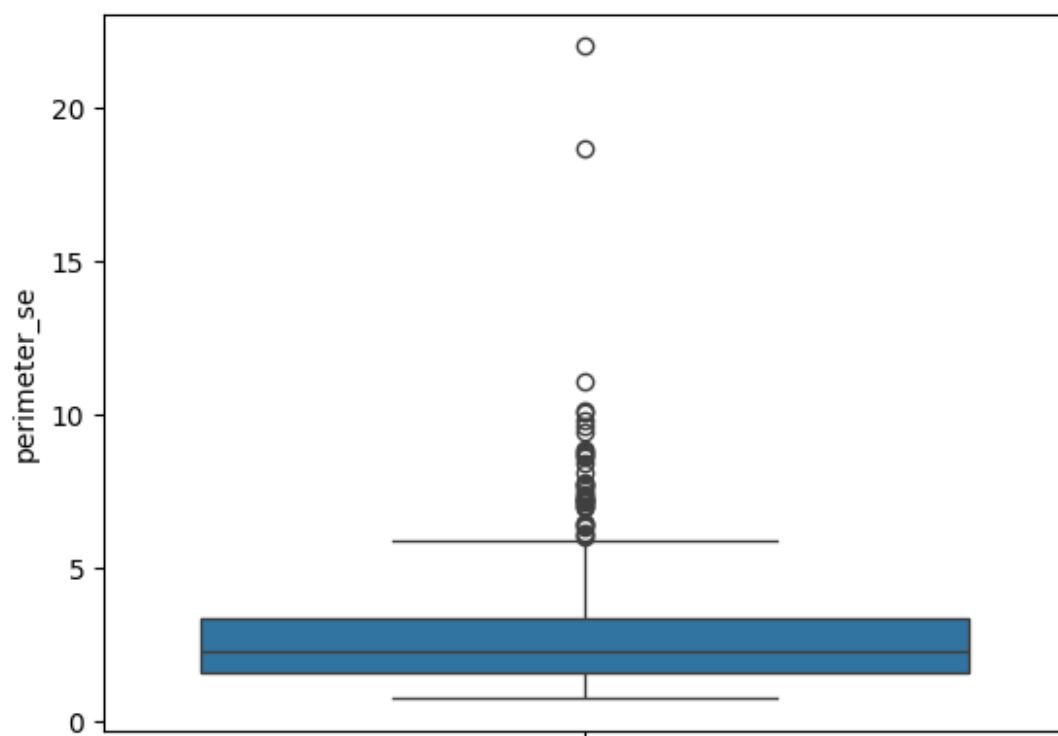
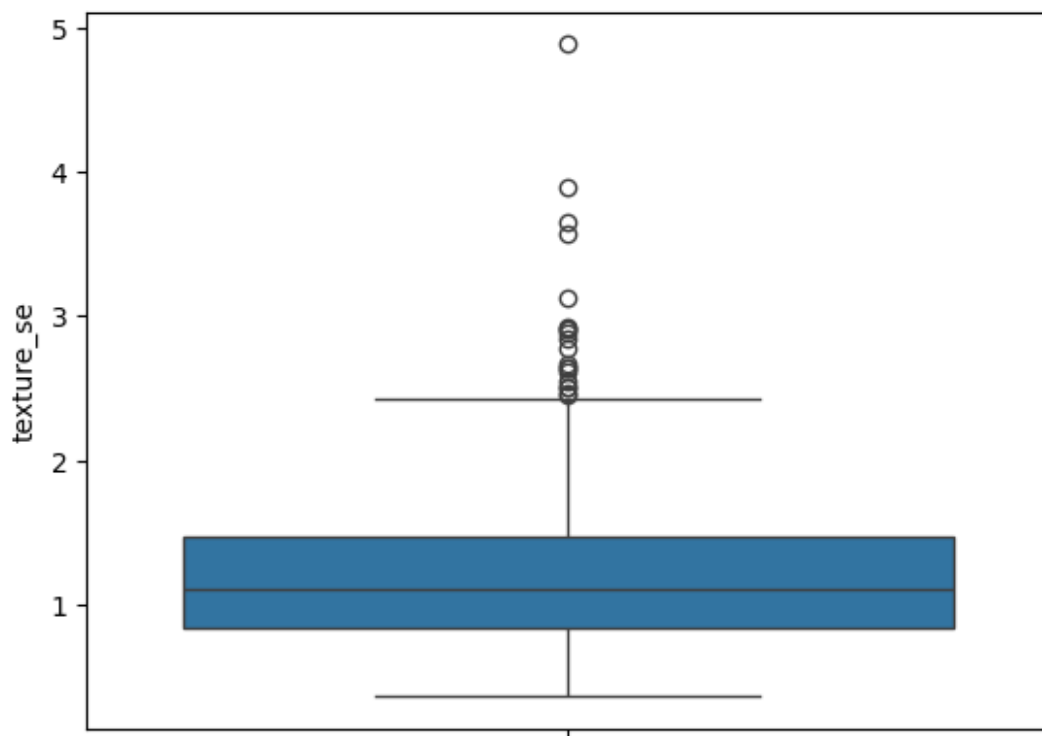


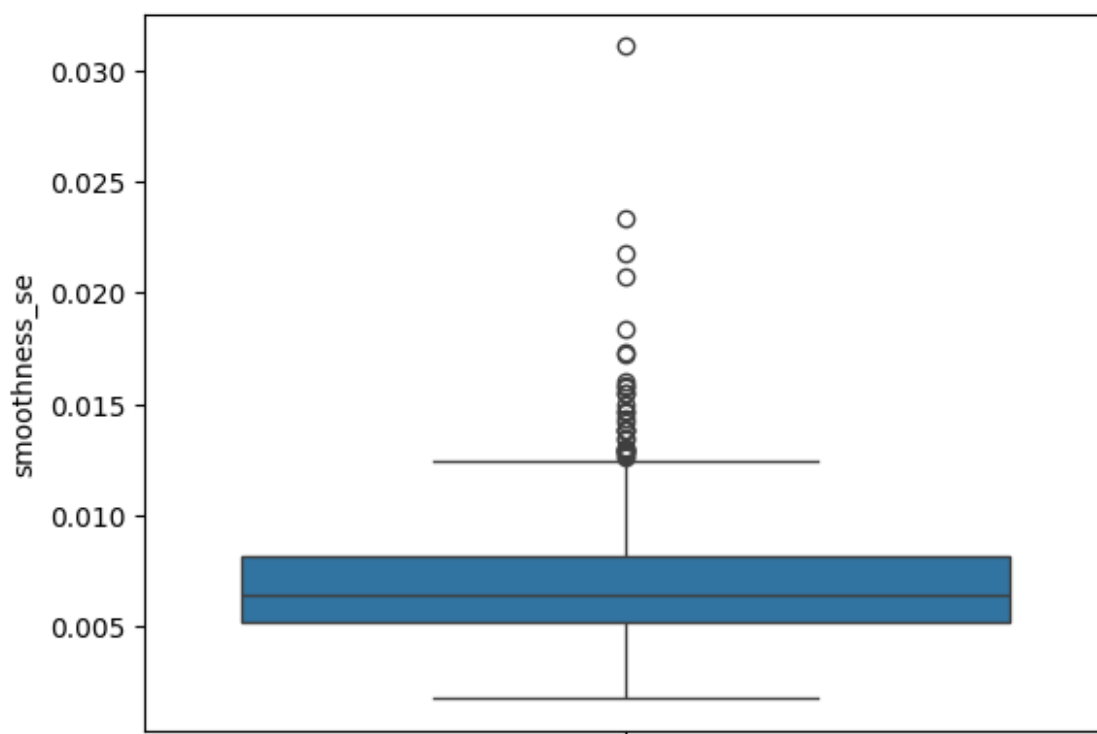
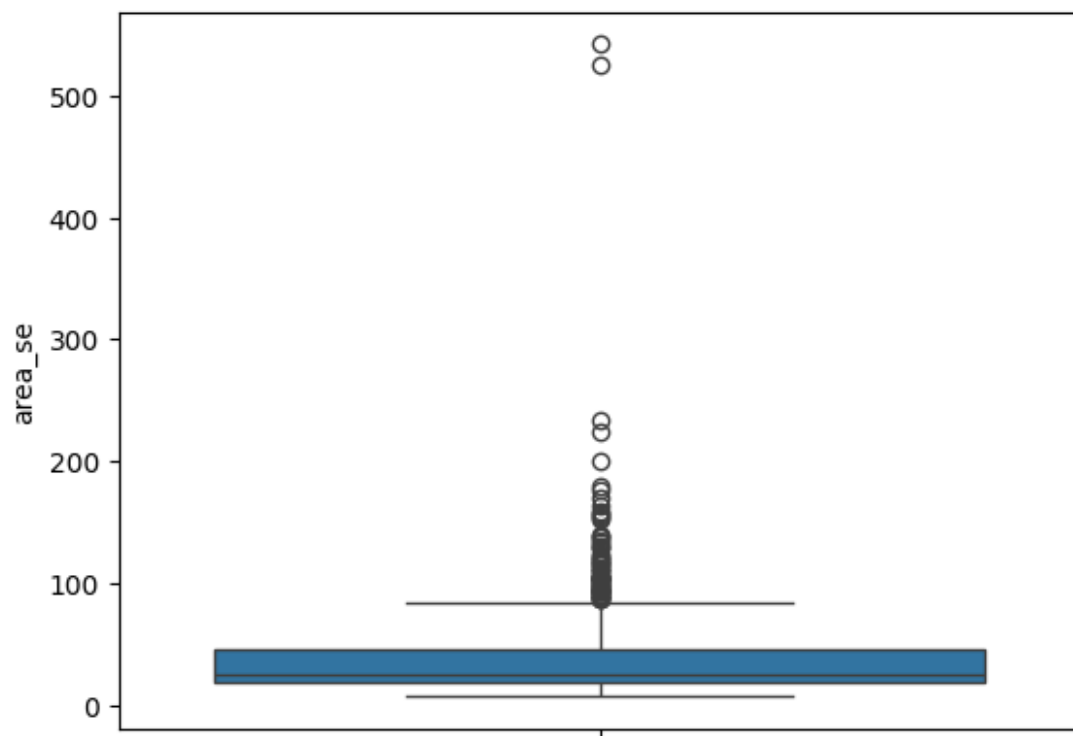


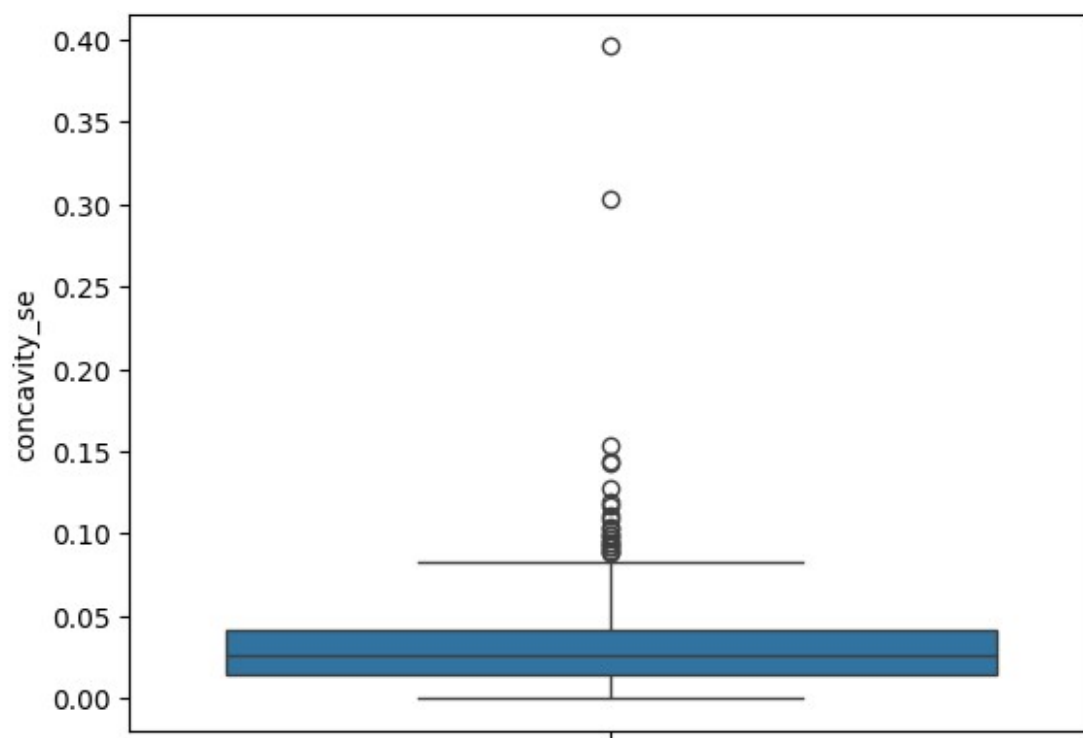
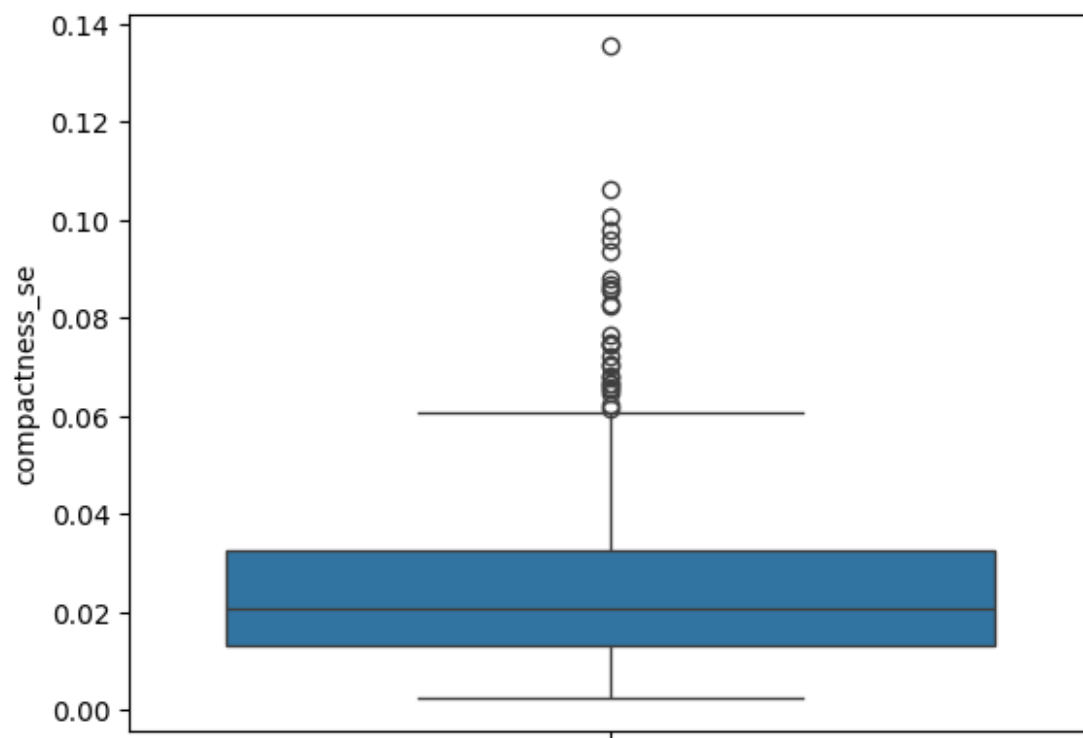


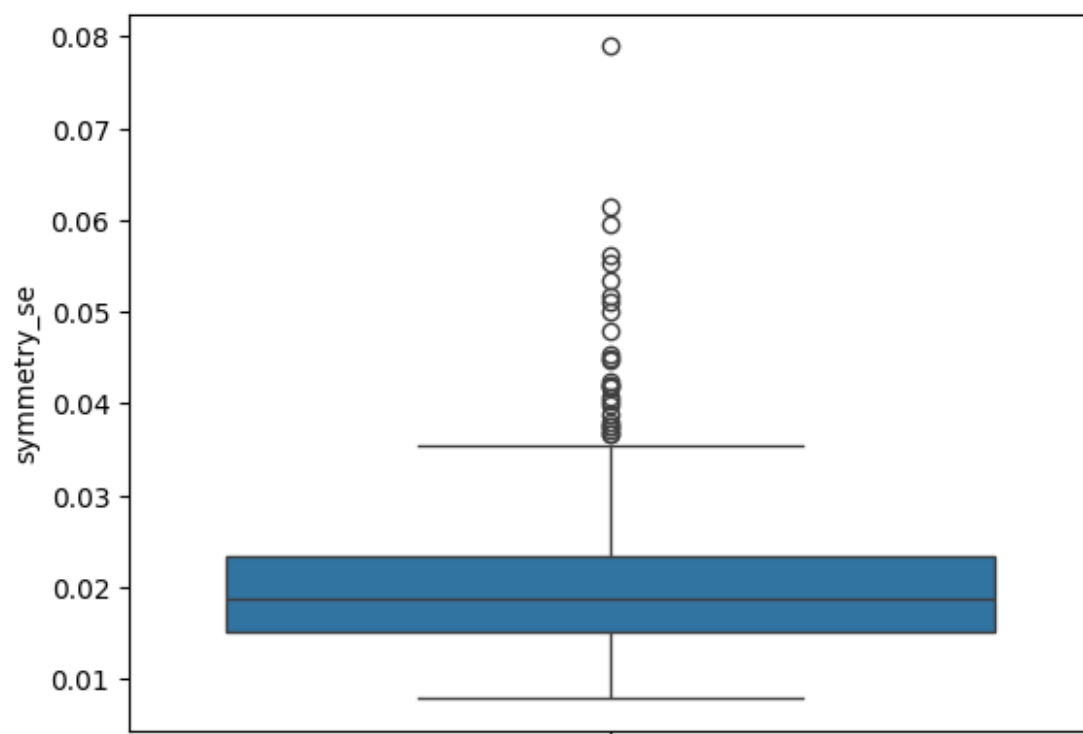
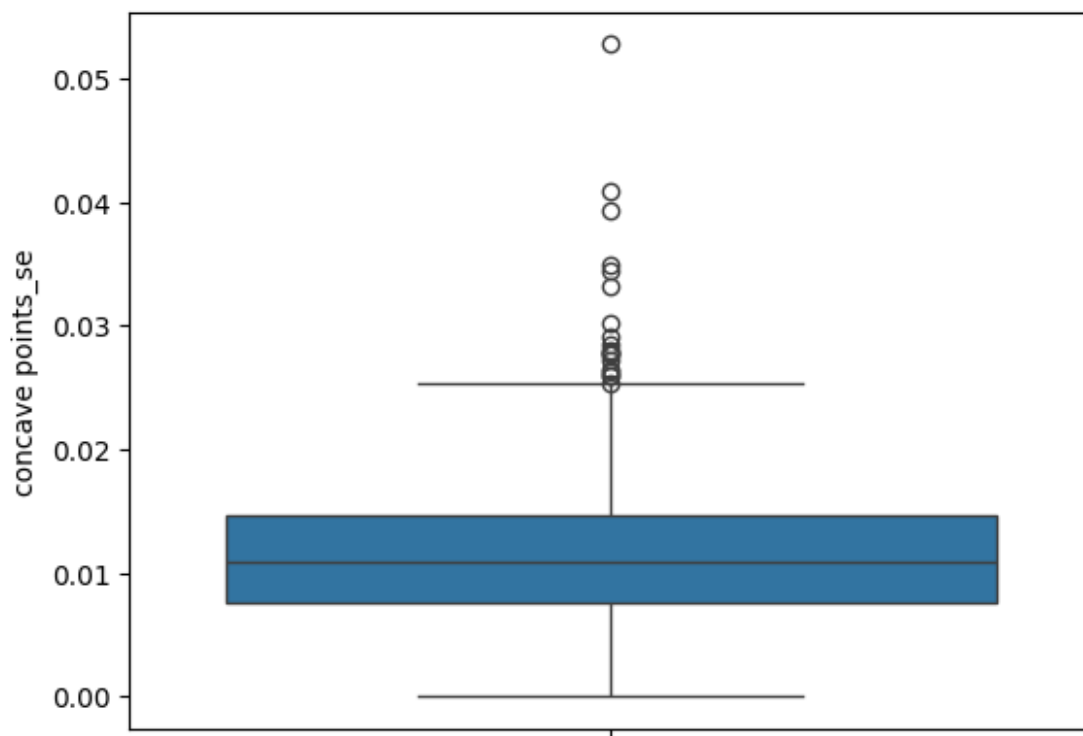


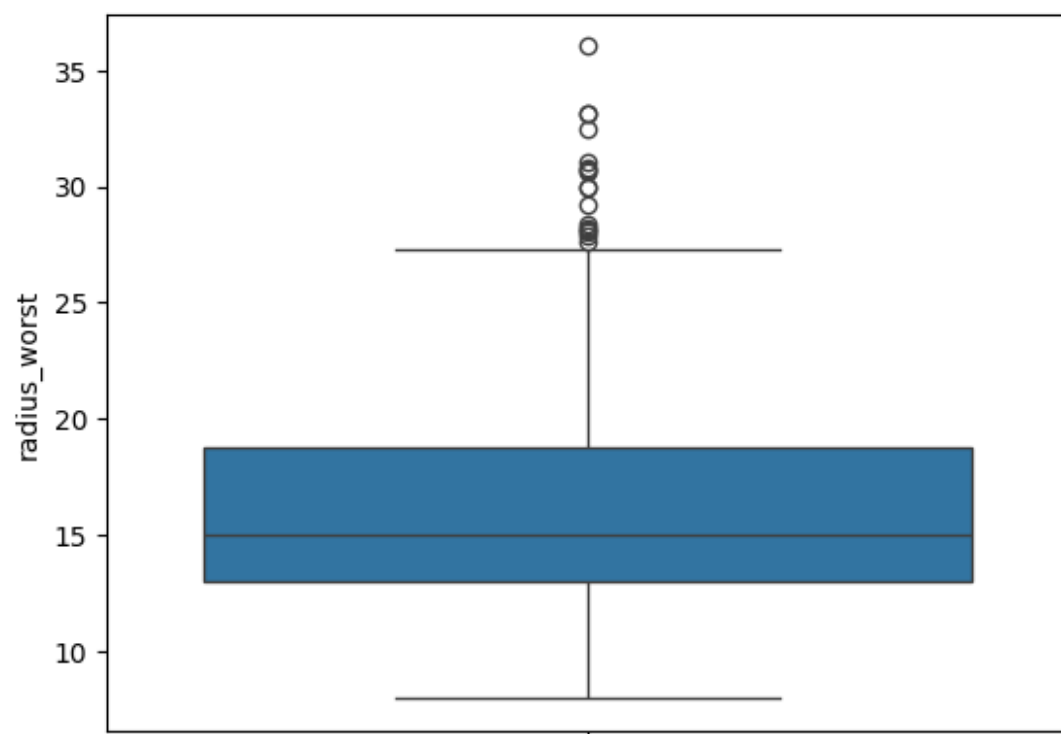
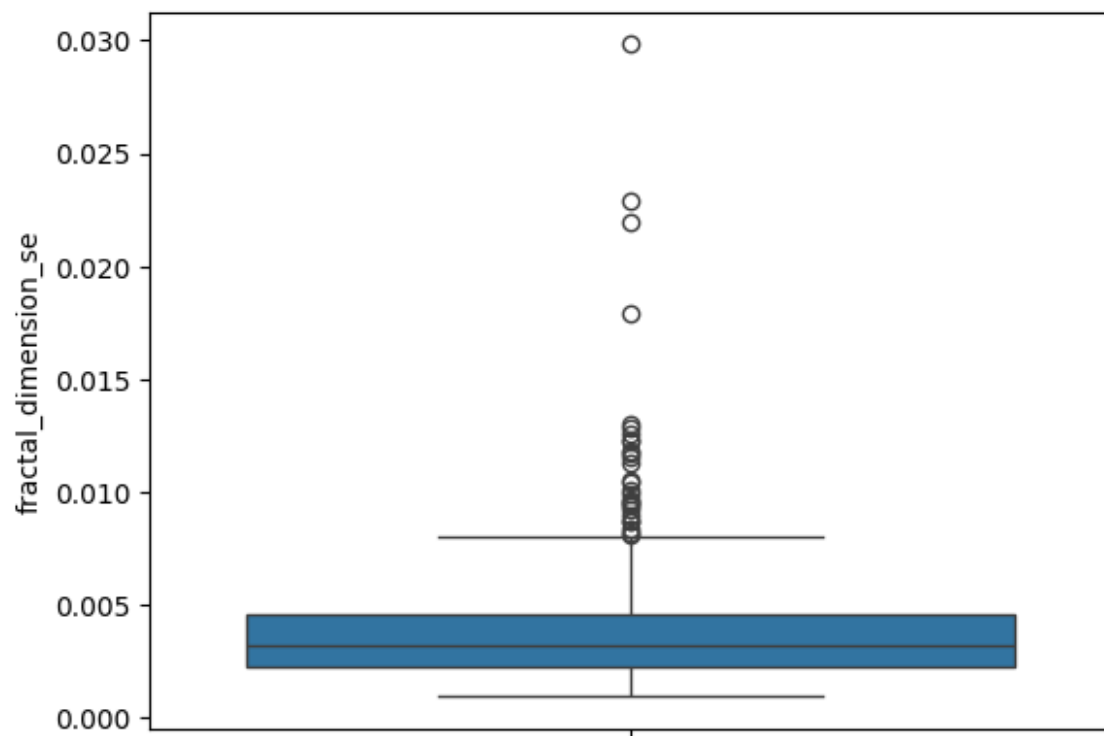


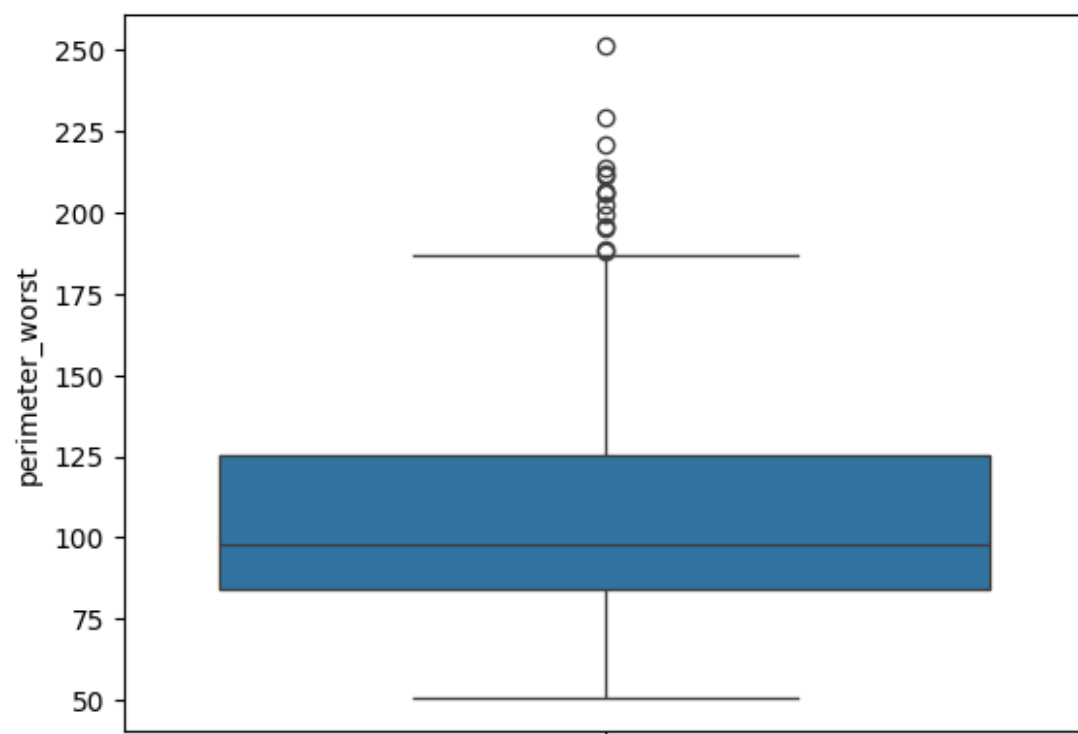
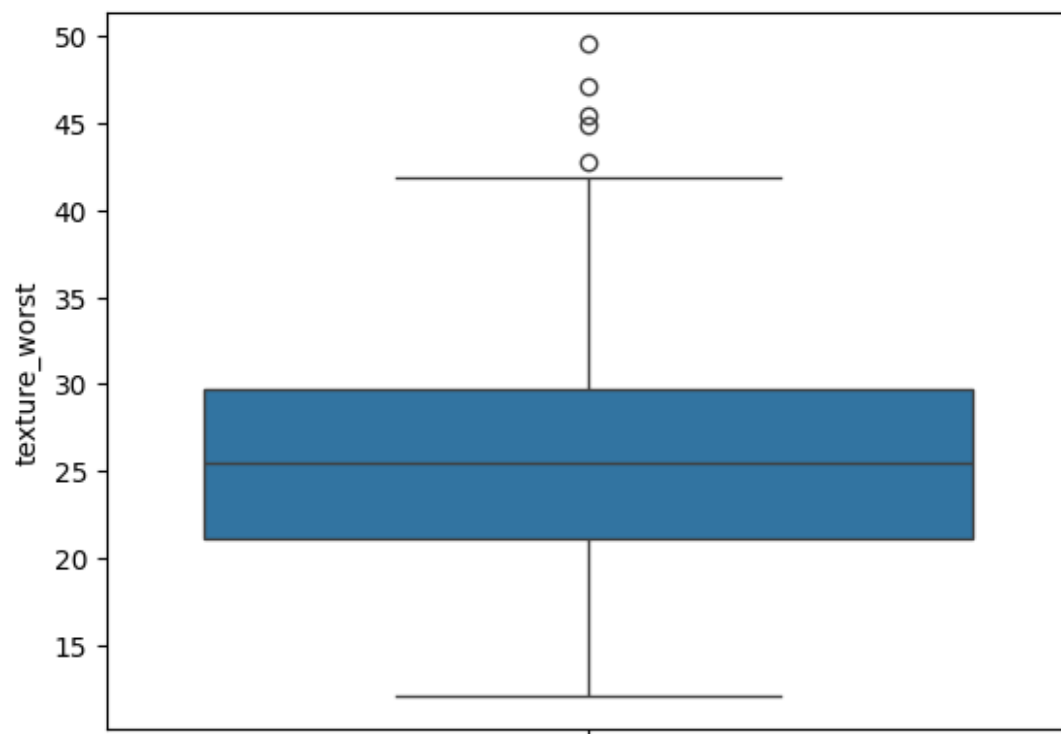


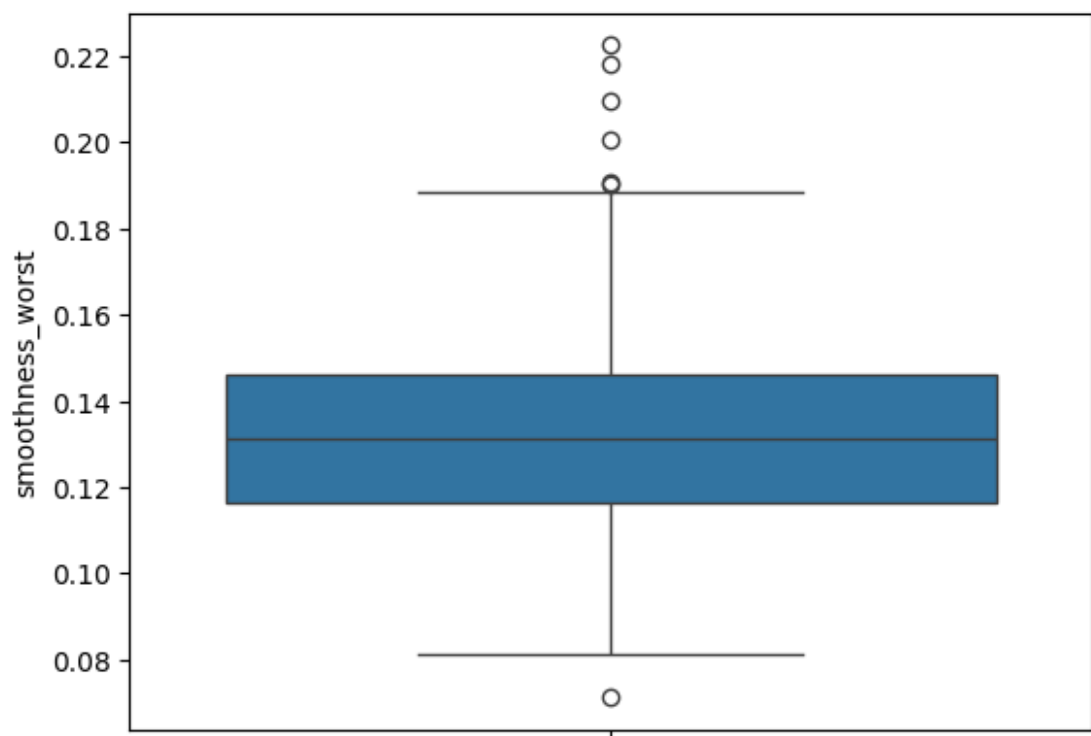
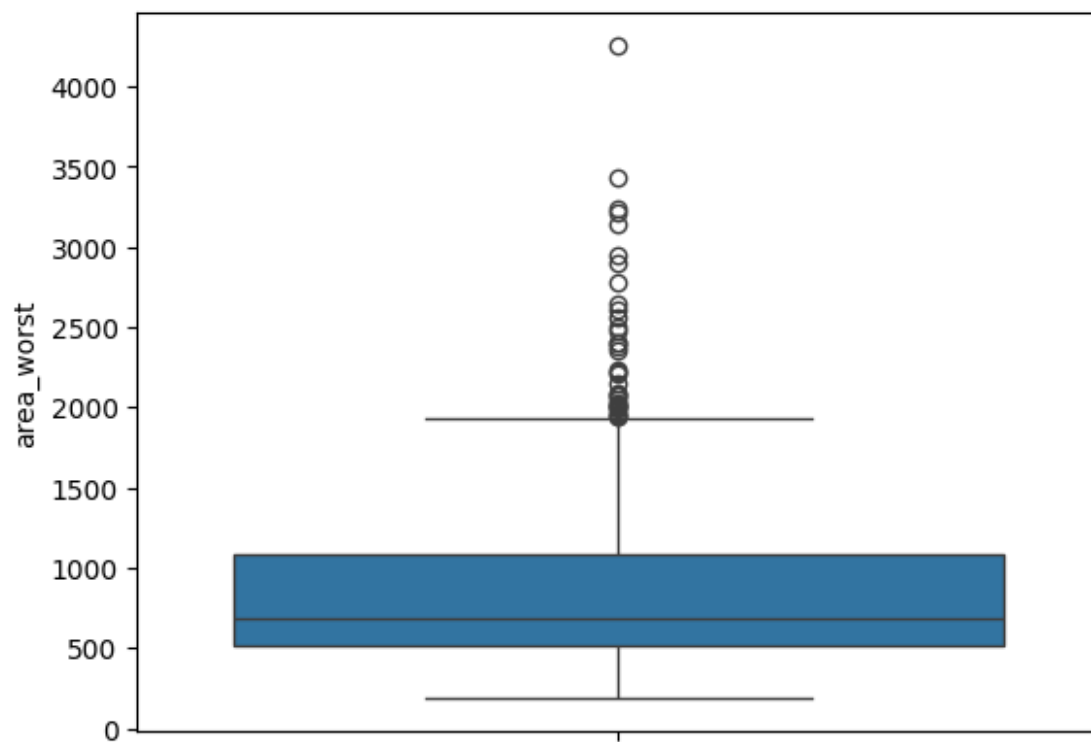


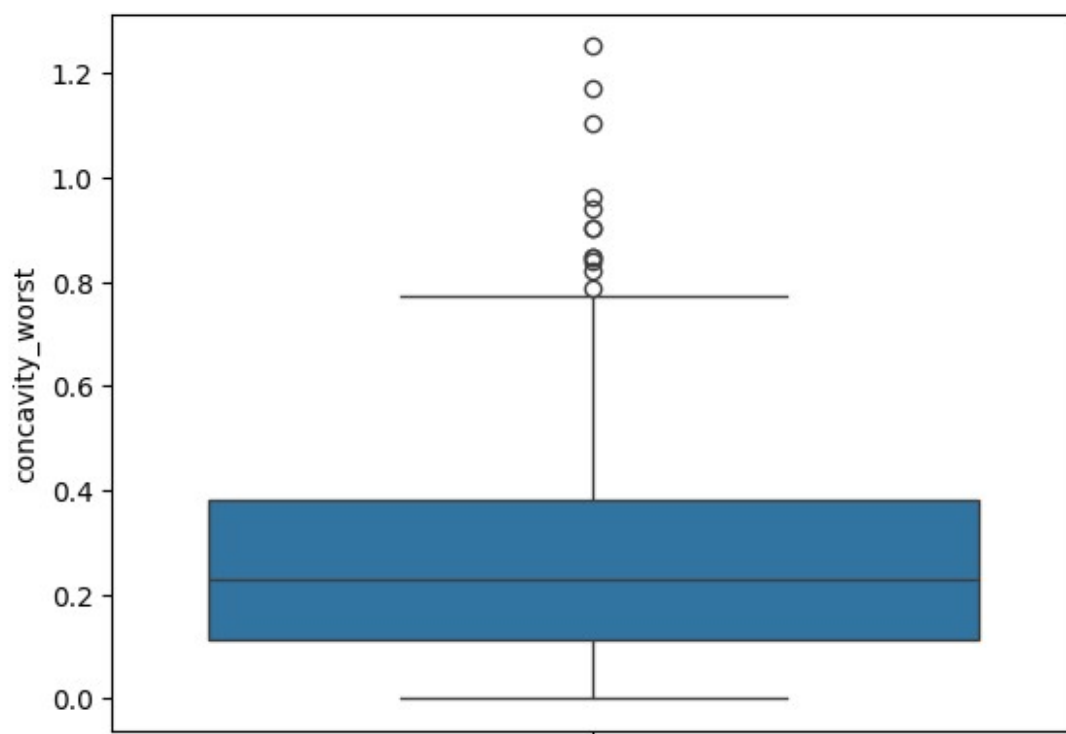
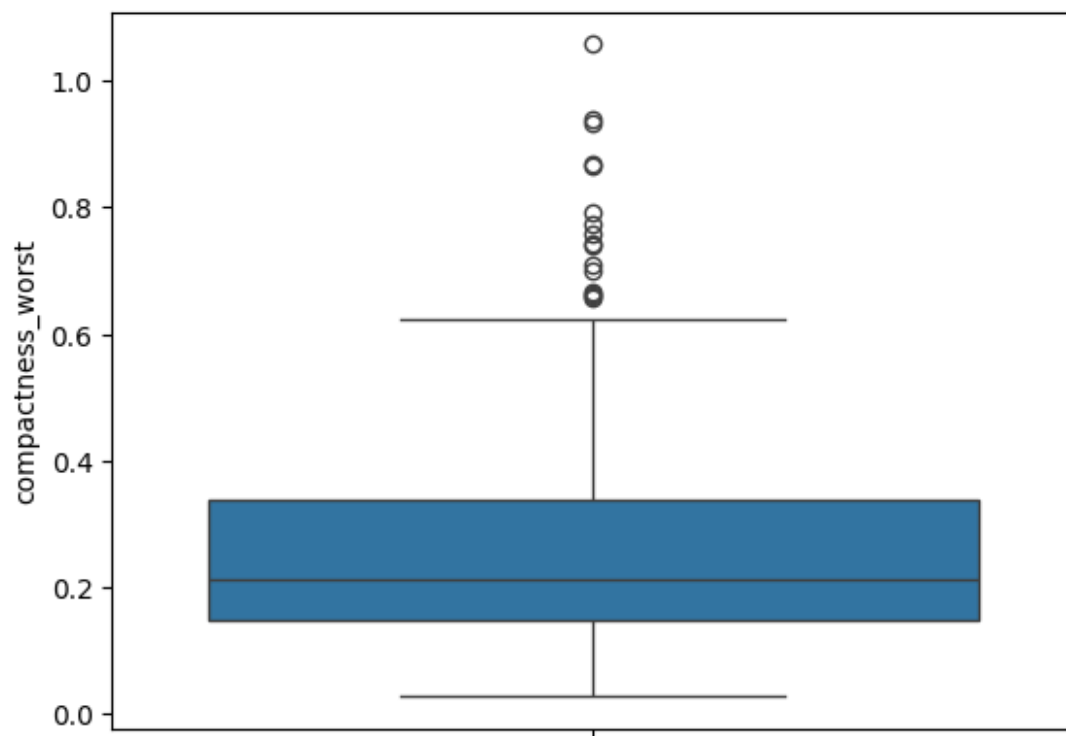


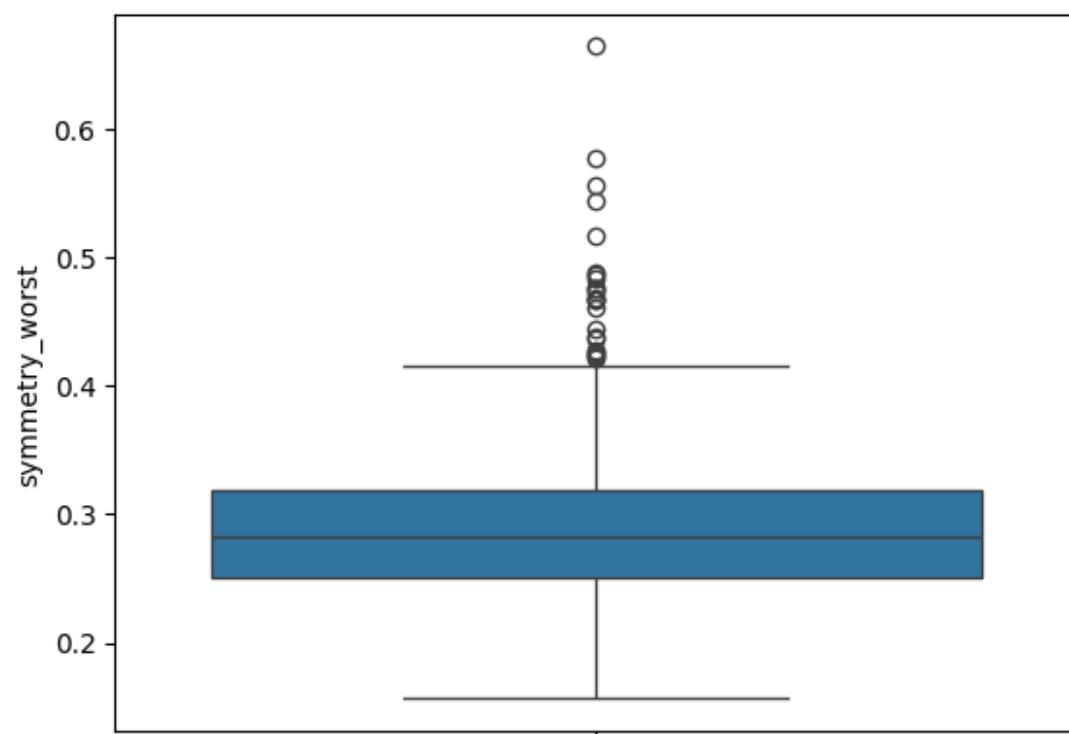
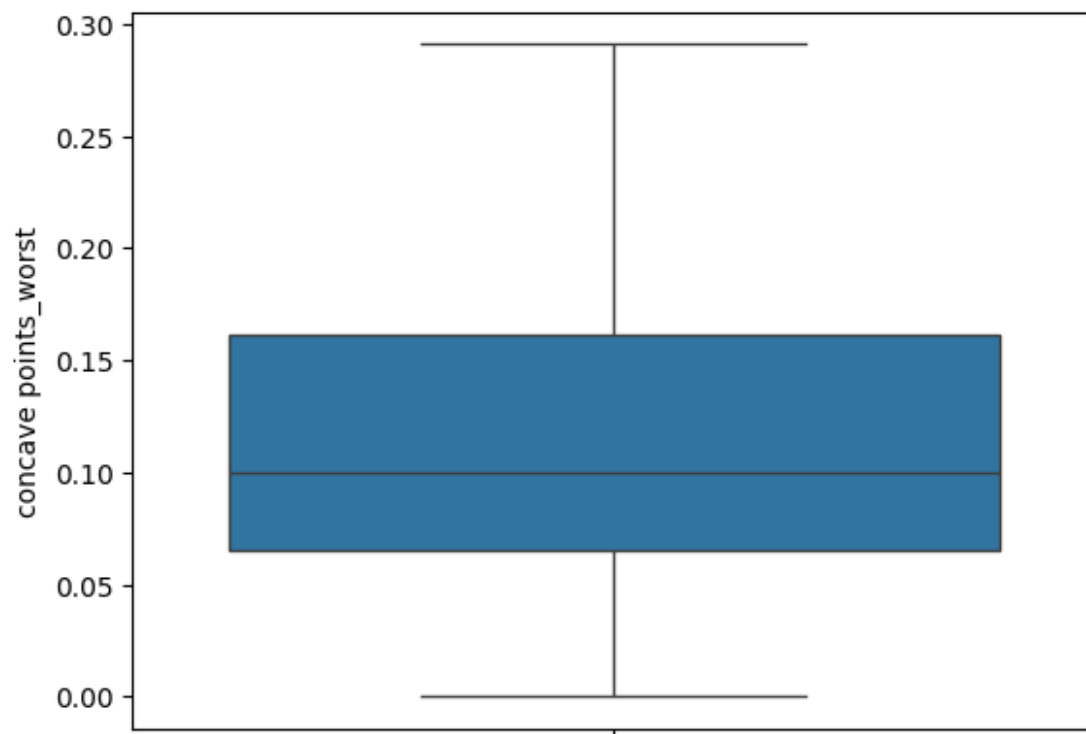


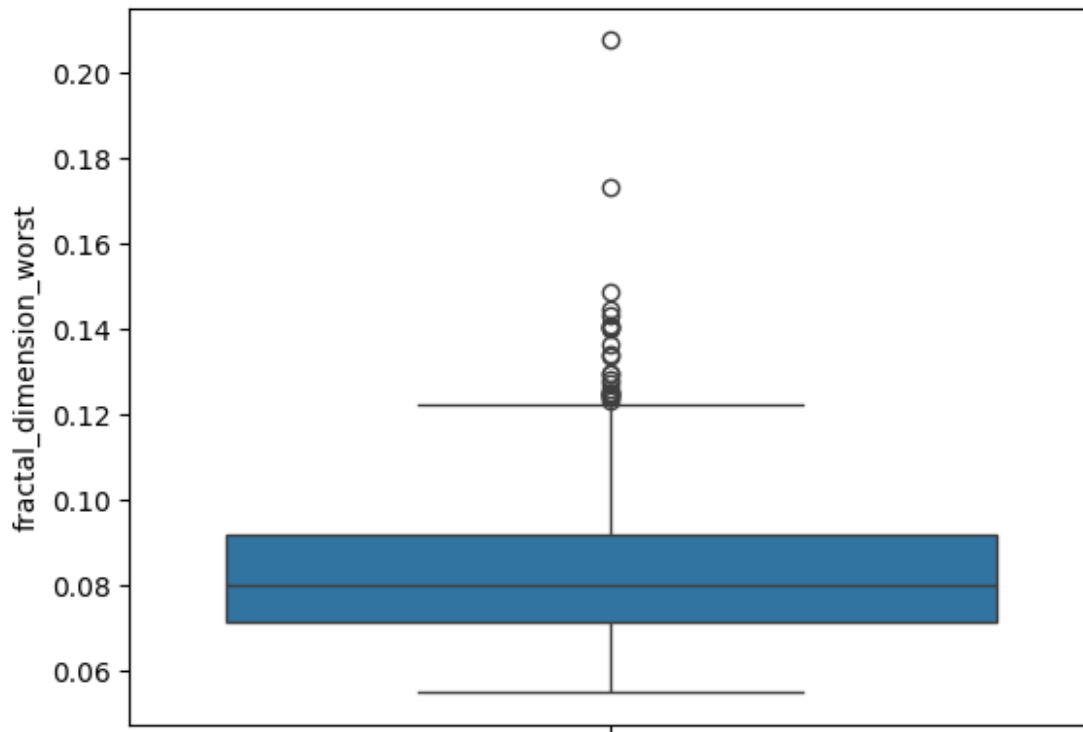












```
# # now want to covert the categorical data to numerical data
# from sklearn.preprocessing import LabelEncoder
# ll=LabelEncoder()
# for i in df.select_dtypes(include='object').columns:
#     df[i]=ll.fit_transform(df[i])
df['diagnosis'] = df['diagnosis'].map({'M':0, 'B':1}).astype(int)
df.head()

{"type": "dataframe", "variable_name": "df"}

df1['diagnosis'].value_counts()

diagnosis
B    357
M    212
Name: count, dtype: int64

%%capture
pip install scikit-learn

df.head()

{"type": "dataframe", "variable_name": "df"}

df1.head()

{"type": "dataframe", "variable_name": "df1"}
```

```

#splitting the data
x=df.iloc[:,1:]
y=df.iloc[:,0]

x.shape
(569, 30)

y.shape
(569,)

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.activations import sigmoid,relu

model=Sequential()

# Input and First Hidden Layer
model.add(Dense(units=64, kernel_initializer='he_uniform',
activation='relu', input_dim=x.shape[1]))

# Additional Hidden Layers
model.add(Dense(units=32, kernel_initializer='he_uniform',
activation='relu'))
model.add(Dense(units=16, kernel_initializer='he_uniform',
activation='relu'))
model.add(Dense(units=4, kernel_initializer='he_uniform',
activation='relu'))

# Output Layer
model.add(Dense(units=1, kernel_initializer='glorot_uniform',
activation='sigmoid'))

/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/
dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

import warnings
warnings.filterwarnings('ignore')

model.summary()

Model: "sequential"

```

Layer (type) Param #	Output Shape
dense (Dense) 1,984	(None, 64)
dense_1 (Dense) 2,080	(None, 32)
dense_2 (Dense) 528	(None, 16)
dense_3 (Dense) 68	(None, 4)
dense_4 (Dense) 5	(None, 1)

Total params: 4,665 (18.22 KB)

Trainable params: 4,665 (18.22 KB)

Non-trainable params: 0 (0.00 B)

```
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
```

```
model.fit(x_train,y_train,epochs=50,batch_size=15,validation_split=0.2
)
```

Epoch 1/50

25/25 ————— 2s 14ms/step - accuracy: 0.6057 - loss: 16.6367 - val_accuracy: 0.8901 - val_loss: 1.3774

Epoch 2/50

25/25 ————— 0s 5ms/step - accuracy: 0.8304 - loss: 0.8799 - val_accuracy: 0.7912 - val_loss: 2.4642

Epoch 3/50

25/25 ————— 0s 5ms/step - accuracy: 0.7713 - loss: 1.9841 - val_accuracy: 0.8901 - val_loss: 1.6126

Epoch 4/50

25/25 ————— 0s 6ms/step - accuracy: 0.8665 - loss: 1.0008 - val_accuracy: 0.8681 - val_loss: 0.6029

Epoch 5/50

25/25 ————— 0s 5ms/step - accuracy: 0.8796 - loss:

0.7490 - val_accuracy: 0.8901 - val_loss: 0.5467
Epoch 6/50
25/25 _____ 0s 5ms/step - accuracy: 0.8937 - loss:
0.3883 - val_accuracy: 0.9011 - val_loss: 0.5235
Epoch 7/50
25/25 _____ 0s 5ms/step - accuracy: 0.8726 - loss:
0.6394 - val_accuracy: 0.7143 - val_loss: 1.4407
Epoch 8/50
25/25 _____ 0s 5ms/step - accuracy: 0.8824 - loss:
0.5353 - val_accuracy: 0.9011 - val_loss: 0.6033
Epoch 9/50
25/25 _____ 0s 5ms/step - accuracy: 0.8831 - loss:
0.7873 - val_accuracy: 0.7802 - val_loss: 2.2959
Epoch 10/50
25/25 _____ 0s 5ms/step - accuracy: 0.8679 - loss:
1.0563 - val_accuracy: 0.8462 - val_loss: 1.1299
Epoch 11/50
25/25 _____ 0s 5ms/step - accuracy: 0.9171 - loss:
0.5023 - val_accuracy: 0.8462 - val_loss: 0.4903
Epoch 12/50
25/25 _____ 0s 5ms/step - accuracy: 0.9204 - loss:
0.4190 - val_accuracy: 0.8901 - val_loss: 0.3914
Epoch 13/50
25/25 _____ 0s 6ms/step - accuracy: 0.9066 - loss:
0.3257 - val_accuracy: 0.9011 - val_loss: 0.3494
Epoch 14/50
25/25 _____ 0s 5ms/step - accuracy: 0.8657 - loss:
0.4947 - val_accuracy: 0.8022 - val_loss: 0.7467
Epoch 15/50
25/25 _____ 0s 5ms/step - accuracy: 0.8898 - loss:
0.5113 - val_accuracy: 0.8901 - val_loss: 0.2957
Epoch 16/50
25/25 _____ 0s 5ms/step - accuracy: 0.8980 - loss:
0.3600 - val_accuracy: 0.8791 - val_loss: 0.2840
Epoch 17/50
25/25 _____ 0s 6ms/step - accuracy: 0.8863 - loss:
0.5521 - val_accuracy: 0.9121 - val_loss: 0.3912
Epoch 18/50
25/25 _____ 0s 5ms/step - accuracy: 0.9069 - loss:
0.3996 - val_accuracy: 0.9121 - val_loss: 0.3467
Epoch 19/50
25/25 _____ 0s 5ms/step - accuracy: 0.8807 - loss:
0.4319 - val_accuracy: 0.9121 - val_loss: 0.3539
Epoch 20/50
25/25 _____ 0s 5ms/step - accuracy: 0.9481 - loss:
0.1855 - val_accuracy: 0.7912 - val_loss: 0.6098
Epoch 21/50
25/25 _____ 0s 5ms/step - accuracy: 0.8661 - loss:
0.5046 - val_accuracy: 0.8901 - val_loss: 0.2461

Epoch 22/50
25/25 _____ 0s 5ms/step - accuracy: 0.8244 - loss: 0.8817 - val_accuracy: 0.9121 - val_loss: 0.5732

Epoch 23/50
25/25 _____ 0s 6ms/step - accuracy: 0.8347 - loss: 2.0890 - val_accuracy: 0.8791 - val_loss: 0.8122

Epoch 24/50
25/25 _____ 0s 5ms/step - accuracy: 0.9211 - loss: 0.5978 - val_accuracy: 0.9121 - val_loss: 0.5085

Epoch 25/50
25/25 _____ 0s 5ms/step - accuracy: 0.8955 - loss: 0.6687 - val_accuracy: 0.9011 - val_loss: 0.8348

Epoch 26/50
25/25 _____ 0s 5ms/step - accuracy: 0.9134 - loss: 0.5367 - val_accuracy: 0.9121 - val_loss: 0.4162

Epoch 27/50
25/25 _____ 0s 5ms/step - accuracy: 0.8916 - loss: 0.4908 - val_accuracy: 0.9231 - val_loss: 0.3445

Epoch 28/50
25/25 _____ 0s 5ms/step - accuracy: 0.9368 - loss: 0.3659 - val_accuracy: 0.7582 - val_loss: 0.8691

Epoch 29/50
25/25 _____ 0s 6ms/step - accuracy: 0.8762 - loss: 0.5248 - val_accuracy: 0.9011 - val_loss: 0.3085

Epoch 30/50
25/25 _____ 0s 5ms/step - accuracy: 0.8871 - loss: 0.3925 - val_accuracy: 0.9121 - val_loss: 0.4109

Epoch 31/50
25/25 _____ 0s 5ms/step - accuracy: 0.9108 - loss: 0.4035 - val_accuracy: 0.9121 - val_loss: 0.4714

Epoch 32/50
25/25 _____ 0s 5ms/step - accuracy: 0.9144 - loss: 0.4768 - val_accuracy: 0.7912 - val_loss: 2.2719

Epoch 33/50
25/25 _____ 0s 5ms/step - accuracy: 0.8093 - loss: 1.5141 - val_accuracy: 0.7912 - val_loss: 2.5112

Epoch 34/50
25/25 _____ 0s 5ms/step - accuracy: 0.8375 - loss: 1.5030 - val_accuracy: 0.8901 - val_loss: 0.8281

Epoch 35/50
25/25 _____ 0s 5ms/step - accuracy: 0.8695 - loss: 1.1212 - val_accuracy: 0.8791 - val_loss: 0.6701

Epoch 36/50
25/25 _____ 0s 5ms/step - accuracy: 0.9277 - loss: 0.4354 - val_accuracy: 0.9121 - val_loss: 0.6350

Epoch 37/50
25/25 _____ 0s 5ms/step - accuracy: 0.8983 - loss: 0.6776 - val_accuracy: 0.9121 - val_loss: 0.4559

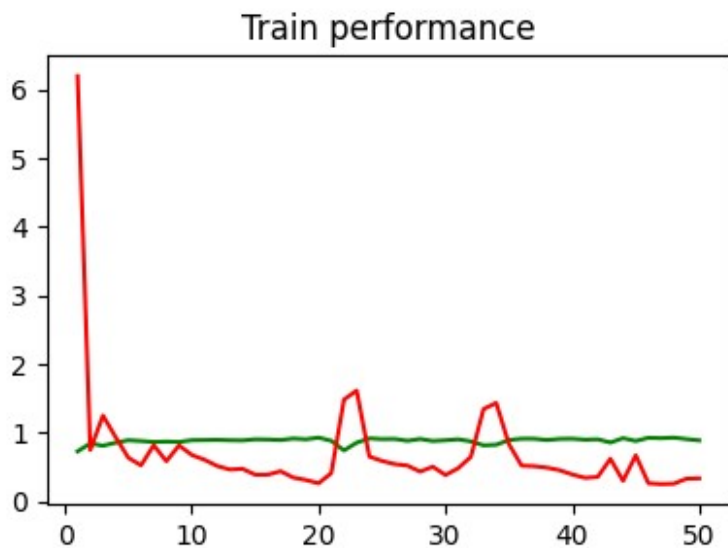
Epoch 38/50

```
25/25 _____ 0s 5ms/step - accuracy: 0.8805 - loss:
0.5790 - val_accuracy: 0.9011 - val_loss: 0.3502
Epoch 39/50
25/25 _____ 0s 7ms/step - accuracy: 0.9029 - loss:
0.4774 - val_accuracy: 0.7473 - val_loss: 0.8482
Epoch 40/50
25/25 _____ 0s 8ms/step - accuracy: 0.8865 - loss:
0.5034 - val_accuracy: 0.9011 - val_loss: 0.2842
Epoch 41/50
25/25 _____ 0s 8ms/step - accuracy: 0.8827 - loss:
0.4353 - val_accuracy: 0.9011 - val_loss: 0.5044
Epoch 42/50
25/25 _____ 0s 8ms/step - accuracy: 0.8960 - loss:
0.4129 - val_accuracy: 0.8242 - val_loss: 0.9577
Epoch 43/50
25/25 _____ 0s 8ms/step - accuracy: 0.8283 - loss:
0.7322 - val_accuracy: 0.9121 - val_loss: 0.2633
Epoch 44/50
25/25 _____ 0s 7ms/step - accuracy: 0.9410 - loss:
0.2096 - val_accuracy: 0.9011 - val_loss: 0.2493
Epoch 45/50
25/25 _____ 0s 7ms/step - accuracy: 0.8873 - loss:
0.5876 - val_accuracy: 0.9121 - val_loss: 0.2640
Epoch 46/50
25/25 _____ 0s 9ms/step - accuracy: 0.9177 - loss:
0.3069 - val_accuracy: 0.9011 - val_loss: 0.2657
Epoch 47/50
25/25 _____ 0s 7ms/step - accuracy: 0.9382 - loss:
0.2370 - val_accuracy: 0.9341 - val_loss: 0.2729
Epoch 48/50
25/25 _____ 0s 5ms/step - accuracy: 0.9407 - loss:
0.2350 - val_accuracy: 0.9121 - val_loss: 0.2822
Epoch 49/50
25/25 _____ 0s 5ms/step - accuracy: 0.9209 - loss:
0.2899 - val_accuracy: 0.9231 - val_loss: 0.3189
Epoch 50/50
25/25 _____ 0s 5ms/step - accuracy: 0.9227 - loss:
0.2153 - val_accuracy: 0.8571 - val_loss: 0.4996
```

```
<keras.src.callbacks.history.History at 0x7c09adba0950>
```

```
plt.figure(figsize=(10,3))
plt.subplot(1,2,1)
plt.title('Train performance')
plt.plot(np.arange(1,51),model.history.history['accuracy'],color='g',label='train_acc')
plt.plot(np.arange(1,51),model.history.history['loss'],color='r',label='train_loss')
```

```
[<matplotlib.lines.Line2D at 0x7c093e5ad410>]
```



```
plt.figure(figsize=(10,3))
plt.subplot(1,2,1)
plt.title('Validation performance')
plt.plot(np.arange(1,51),model.history.history['val_accuracy'],color='g',label='val_acc')
plt.plot(np.arange(1,51),model.history.history['val_loss'],color='r',label='val_loss')
```

[<matplotlib.lines.Line2D at 0x7c093ff50050>]

